Stream Data

If you want to stream data that can be structured as JSON, you should Stream JSON Lines.

But if you want to **stream pure binary data** or strings, here's how you can do it.

Info

Added in FastAPI 0.134.0.

Use Cases

You could use this if you want to stream pure strings, for example directly from the output of an **AI LLM** service.

You could also use it to stream **large binary files**, where you stream each chunk of data as you read it, without having to read it all in memory at once.

You could also stream **video** or **audio** this way, it could even be generated as you process and send it.

A `StreamingResponse` with `yield`

If you declare a `response_class=StreamingResponse` in your *path operation function*, you can use `yield` to send each chunk of data in turn.

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()


message = """
Rick: (stumbles in drunkenly, and turns on the lights) Morty! You gotta come on. You got--... you gotta come with me.
Morty: (rubs his eyes) What, Rick? What's going on?
Rick: I got a surprise for you, Morty.
Morty: It's the middle of the night. What are you talking about?
Rick: (spills alcohol on Morty's bed) Come on, I got a surprise for you. (drags Morty by the ankle) Come on, hurry up. (pulls Morty out of his bed and
into the hall)
Morty: Ow! Ow! You're tugging me too hard!
Rick: We gotta go, gotta get outta here, come on. Got a surprise for you Morty.
"""


@app.get("/story/stream", response_class=StreamingResponse)
async def stream_story() -> AsyncIterable[str]:
    for line in message.splitlines():
        yield line

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()


message = """
Rick: (stumbles in drunkenly, and turns on the lights) Morty! You gotta come on. You got--... you gotta come with me.
Morty: (rubs his eyes) What, Rick? What's going on?
Rick: I got a surprise for you, Morty.
Morty: It's the middle of the night. What are you talking about?
Rick: (spills alcohol on Morty's bed) Come on, I got a surprise for you. (drags Morty by the ankle) Come on, hurry up. (pulls Morty out of his bed and into the hall)
Morty: Ow! Ow! You're tugging me too hard!
Rick: We gotta go, gotta get outta here, come on. Got a surprise for you Morty.
"""


@app.get("/story/stream", response_class=StreamingResponse)
async def stream_story() -> AsyncIterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async", response_class=StreamingResponse)
def stream_story_no_async() -> Iterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-annotation", response_class=StreamingResponse)
async def stream_story_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async-no-annotation", response_class=StreamingResponse)
def stream_story_no_async_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-bytes", response_class=StreamingResponse)
async def stream_story_bytes() -> AsyncIterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-bytes", response_class=StreamingResponse)
def stream_story_no_async_bytes() -> Iterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-annotation-bytes", response_class=StreamingResponse)
async def stream_story_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-no-annotation-bytes", response_class=StreamingResponse)
def stream_story_no_async_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")
```

FastAPI will give each chunk of data to the `StreamingResponse` as is, it won't try to convert it to JSON or anything similar.

Non-async *path operation functions*

You can also use regular `def` functions (without `async`), and use `yield` the same way.

Python 3.10+

```python
# Code above omitted 👆

@app.get("/story/stream-no-async", response_class=StreamingResponse)
def stream_story_no_async() -> Iterable[str]:
    for line in message.splitlines():
        yield line

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()


message = """
Rick: (stumbles in drunkenly, and turns on the lights) Morty! You gotta come on. You got--... you gotta come with me.
Morty: (rubs his eyes) What, Rick? What's going on?
Rick: I got a surprise for you, Morty.
Morty: It's the middle of the night. What are you talking about?
Rick: (spills alcohol on Morty's bed) Come on, I got a surprise for you. (drags Morty by the ankle) Come on, hurry up. (pulls Morty out of his bed and into the hall)
Morty: Ow! Ow! You're tugging me too hard!
Rick: We gotta go, gotta get outta here, come on. Got a surprise for you Morty.
"""


@app.get("/story/stream", response_class=StreamingResponse)
async def stream_story() -> AsyncIterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async", response_class=StreamingResponse)
def stream_story_no_async() -> Iterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-annotation", response_class=StreamingResponse)
async def stream_story_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async-no-annotation", response_class=StreamingResponse)
def stream_story_no_async_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-bytes", response_class=StreamingResponse)
async def stream_story_bytes() -> AsyncIterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-bytes", response_class=StreamingResponse)
def stream_story_no_async_bytes() -> Iterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-annotation-bytes", response_class=StreamingResponse)
async def stream_story_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-no-annotation-bytes", response_class=StreamingResponse)
def stream_story_no_async_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")
```

No Annotation

You don't really need to declare the return type annotation for streaming binary data.

As FastAPI will not try to convert the data to JSON with Pydantic or serialize it in any way, in this case, the type annotation is only for your editor and tools to use, it won't be used by FastAPI.

Python 3.10+

```python
# Code above omitted 👆

@app.get("/story/stream-no-annotation", response_class=StreamingResponse)
async def stream_story_no_annotation():
    for line in message.splitlines():
        yield line

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()


message = """
Rick: (stumbles in drunkenly, and turns on the lights) Morty! You gotta come on. You got--... you gotta come with me.
Morty: (rubs his eyes) What, Rick? What's going on?
Rick: I got a surprise for you, Morty.
Morty: It's the middle of the night. What are you talking about?
Rick: (spills alcohol on Morty's bed) Come on, I got a surprise for you. (drags Morty by the ankle) Come on, hurry up. (pulls Morty out of his bed and into the hall)
Morty: Ow! Ow! You're tugging me too hard!
Rick: We gotta go, gotta get outta here, come on. Got a surprise for you Morty.
"""


@app.get("/story/stream", response_class=StreamingResponse)
async def stream_story() -> AsyncIterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async", response_class=StreamingResponse)
def stream_story_no_async() -> Iterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-annotation", response_class=StreamingResponse)
async def stream_story_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async-no-annotation", response_class=StreamingResponse)
def stream_story_no_async_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-bytes", response_class=StreamingResponse)
async def stream_story_bytes() -> AsyncIterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-bytes", response_class=StreamingResponse)
def stream_story_no_async_bytes() -> Iterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-annotation-bytes", response_class=StreamingResponse)
async def stream_story_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-no-annotation-bytes", response_class=StreamingResponse)
def stream_story_no_async_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")
```

This also means that with `StreamingResponse` you have the **freedom** and **responsibility** to produce and encode the data bytes exactly as you need them to be sent, independent of the type annotations. 🤓

Stream Bytes

One of the main use cases would be to stream `bytes` instead of strings, you can of course do it.

Python 3.10+

```python
# Code above omitted 👆

@app.get("/story/stream-bytes", response_class=StreamingResponse)
async def stream_story_bytes() -> AsyncIterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()


message = """
Rick: (stumbles in drunkenly, and turns on the lights) Morty! You gotta come on. You got--... you gotta come with me.
Morty: (rubs his eyes) What, Rick? What's going on?
Rick: I got a surprise for you, Morty.
Morty: It's the middle of the night. What are you talking about?
Rick: (spills alcohol on Morty's bed) Come on, I got a surprise for you. (drags Morty by the ankle) Come on, hurry up. (pulls Morty out of his bed and into the hall)
Morty: Ow! Ow! You're tugging me too hard!
Rick: We gotta go, gotta get outta here, come on. Got a surprise for you Morty.
"""


@app.get("/story/stream", response_class=StreamingResponse)
async def stream_story() -> AsyncIterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async", response_class=StreamingResponse)
def stream_story_no_async() -> Iterable[str]:
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-annotation", response_class=StreamingResponse)
async def stream_story_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-no-async-no-annotation", response_class=StreamingResponse)
def stream_story_no_async_no_annotation():
    for line in message.splitlines():
        yield line


@app.get("/story/stream-bytes", response_class=StreamingResponse)
async def stream_story_bytes() -> AsyncIterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-bytes", response_class=StreamingResponse)
def stream_story_no_async_bytes() -> Iterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-annotation-bytes", response_class=StreamingResponse)
async def stream_story_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")


@app.get("/story/stream-no-async-no-annotation-bytes", response_class=StreamingResponse)
def stream_story_no_async_no_annotation_bytes():
    for line in message.splitlines():
        yield line.encode("utf-8")
```

A Custom `PNGStreamingResponse`

In the examples above, the data bytes were streamed, but the response didn't have a `Content-Type` header, so the client didn't know what type of data it was receiving.

You can create a custom sub-class of `StreamingResponse` that sets the `Content-Type` header to the type of data you're streaming.

For example, you can create a `PNGStreamingResponse` that sets the `Content-Type` header to `image/png` using the `media_type` attribute:

Python 3.10+

```python
# Code above omitted 👆

from fastapi.responses import StreamingResponse

# Code here omitted 👈

class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
import base64
from collections.abc import AsyncIterable, Iterable
from io import BytesIO

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

image_base64 =
"iVBORw0KGgoAAAANSUhEUgAAAB0AAAAdCAYAAABWk2cPAAAAbnpUWHRSYXcgcHJvZmlsZSB0eXBlIGV4aWYAAHjadYzRDYAwCET/mcIRDoq0jGOiJm7g+NJK0vjhS4DjIEfHfZ20DKqSrrWZmyFQV5ctRMOLACxglN
binary_image = base64.b64decode(image_base64)


def read_image() -> BytesIO:
    return BytesIO(binary_image)


app = FastAPI()


class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"


@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async", response_class=PNGStreamingResponse)
def stream_image_no_async() -> Iterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-yield-from", response_class=PNGStreamingResponse)
def stream_image_no_async_yield_from() -> Iterable[bytes]:
    with read_image() as image_file:
        yield from image_file


@app.get("/image/stream-no-annotation", response_class=PNGStreamingResponse)
async def stream_image_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-no-annotation", response_class=PNGStreamingResponse)
def stream_image_no_async_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```

Then you can use this new class in `response_class=PNGStreamingResponse` in your *path operation function*:

Python 3.10+

```python
# Code above omitted 👆

@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
import base64
from collections.abc import AsyncIterable, Iterable
from io import BytesIO

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

image_base64 =
"iVBORw0KGgoAAAANSUhEUgAAAB0AAAAdCAYAAABWk2cPAAAAbnpUWHRSYXcgcHJvZmlsZSB0eXBlIGV4aWYAAHjadYzRDYAwCET/mcIRDoq0jGOiJm7g+NJK0vjhS4DjIEfHfZ20DKqSrrWZmyFQV5ctRMOLACxglN
binary_image = base64.b64decode(image_base64)


def read_image() -> BytesIO:
    return BytesIO(binary_image)


app = FastAPI()


class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"


@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async", response_class=PNGStreamingResponse)
def stream_image_no_async() -> Iterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-yield-from", response_class=PNGStreamingResponse)
def stream_image_no_async_yield_from() -> Iterable[bytes]:
    with read_image() as image_file:
        yield from image_file


@app.get("/image/stream-no-annotation", response_class=PNGStreamingResponse)
async def stream_image_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-no-annotation", response_class=PNGStreamingResponse)
def stream_image_no_async_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```

Simulate a File

In this example, we are simulating a file with `io.BytesIO`, which is a file-like object that lives only in memory, but lets us use the same interface.

For example, we can iterate over it to consume its contents, as we could with a file.

Python 3.10+

```python
import base64
from collections.abc import AsyncIterable, Iterable
from io import BytesIO

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

image_base64 =
"iVBORw0KGgoAAAANSUhEUgAAAB0AAAAdCAYAAABWk2cPAAAAbnpUWHRSYXcgcHJvZmlsZSB0eXBlIGV4aWYAAHjadYzRDYAwCET/mcIRDoq0jGOiJm7g+NJK0vjhS4DjIEfHfZ20DKqSrrWZmyFQV5ctRM
binary_image = base64.b64decode(image_base64)


def read_image() -> BytesIO:
    return BytesIO(binary_image)


app = FastAPI()


class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"


@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk

# Code below omitted 👇
```

Full file preview

Python 3.10+

```python
import base64
from collections.abc import AsyncIterable, Iterable
from io import BytesIO

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

image_base64 =
"iVBORw0KGgoAAAANSUhEUgAAAB0AAAAdCAYAAABWk2cPAAAAbnpUWHRSYXcgcHJvZmlsZSB0eXBlIGV4aWYAAHjadYzRDYAwCET/mcIRDoq0jGOiJm7g+NJK0vjhS4DjIEfHfZ20DKqSrrWZmyFQV5ctRMOLACxglN
binary_image = base64.b64decode(image_base64)


def read_image() -> BytesIO:
    return BytesIO(binary_image)


app = FastAPI()


class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"


@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async", response_class=PNGStreamingResponse)
def stream_image_no_async() -> Iterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-yield-from", response_class=PNGStreamingResponse)
def stream_image_no_async_yield_from() -> Iterable[bytes]:
    with read_image() as image_file:
        yield from image_file


@app.get("/image/stream-no-annotation", response_class=PNGStreamingResponse)
async def stream_image_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-no-annotation", response_class=PNGStreamingResponse)
def stream_image_no_async_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```

Technical Details

The other two variables, `image_base64` and `binary_image`, are an image encoded in Base 64, and then converted to bytes, to then pass it to `io.BytesIO`.

Only so that it can live in the same file for this example and you can copy it and run it as is. 🥚

By using a `with` block, we make sure that the file-like object is closed after the generator function (the function with `yield`) is done. So, after it finishes sending the response.

It wouldn't be that important in this specific example because it's a fake in-memory file (with `io.BytesIO`), but with a real file, it would be important to make sure the file is closed after the work with it is done.

Files and Async

In most cases, file-like objects are not compatible with async and await by default.

For example, they don't have an `await file.read()`, or `async for chunk in file`.

And in many cases, reading them would be a blocking operation (that could block the event loop), because they are read from disk or from the network.

Info

The example above is actually an exception, because the `io.BytesIO` object is already in memory, so reading it won't block anything.

But in many cases reading a file or a file-like object would block.

To avoid blocking the event loop, you can simply declare the *path operation function* with regular `def` instead of `async def`, that way FastAPI will run it on a threadpool worker, to avoid blocking the main loop.

Python 3.10+

```python
# Code above omitted 👆

@app.get("/image/stream-no-async", response_class=PNGStreamingResponse)
def stream_image_no_async() -> Iterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```

```
# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
import base64
from collections.abc import AsyncIterable, Iterable
from io import BytesIO

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

image_base64 =
"iVBORw0KGgoAAAANSUhEUgAAAB0AAAAdCAYAAABWk2cPAAAAbnpUWHRSYXcgcHJvZmlsZSB0eXBlIGV4aWYAAHjadYzRDYAwCET/mcIRDoq0jGOiJm7g+NJK0vjhS4DjIEfHfZ20DKqSrrWZmyFQV5ctRMOLACxglN
binary_image = base64.b64decode(image_base64)


def read_image() -> BytesIO:
    return BytesIO(binary_image)


app = FastAPI()


class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"


@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async", response_class=PNGStreamingResponse)
def stream_image_no_async() -> Iterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-yield-from", response_class=PNGStreamingResponse)
def stream_image_no_async_yield_from() -> Iterable[bytes]:
    with read_image() as image_file:
        yield from image_file


@app.get("/image/stream-no-annotation", response_class=PNGStreamingResponse)
async def stream_image_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-no-annotation", response_class=PNGStreamingResponse)
def stream_image_no_async_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```

Tip

If you need to call blocking code from inside of an async function, or an async function from inside of a blocking function, you could use Asyncer, a sibling library to FastAPI.

`yield from`

When you are iterating over something, like a file-like object, and then you are doing `yield` for each item, you could also use `yield from` to yield each item directly and skip the `for` loop.

This is not particular to FastAPI, it's just Python, but it's a nice trick to know. 😎

Python 3.10+

```python
# Code above omitted 👆

@app.get("/image/stream-no-async-yield-from", response_class=PNGStreamingResponse)
def stream_image_no_async_yield_from() -> Iterable[bytes]:
    with read_image() as image_file:
        yield from image_file

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
import base64
from collections.abc import AsyncIterable, Iterable
from io import BytesIO

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

image_base64 =
"iVBORw0KGgoAAAANSUhEUgAAAB0AAAAdCAYAAABWk2cPAAAAbnpUWHRSYXcgcHJvZmlsZSB0eXBlIGV4aWYAAHjadYzRDYAwCET/mcIRDoq0jGOiJm7g+NJK0vjhS4DjIEfHfZ20DKqSrrWZmyFQV5ctRMOLACxglN
binary_image = base64.b64decode(image_base64)


def read_image() -> BytesIO:
    return BytesIO(binary_image)


app = FastAPI()


class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"


@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async", response_class=PNGStreamingResponse)
def stream_image_no_async() -> Iterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-yield-from", response_class=PNGStreamingResponse)
def stream_image_no_async_yield_from() -> Iterable[bytes]:
    with read_image() as image_file:
        yield from image_file


@app.get("/image/stream-no-annotation", response_class=PNGStreamingResponse)
async def stream_image_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk


@app.get("/image/stream-no-async-no-annotation", response_class=PNGStreamingResponse)
def stream_image_no_async_no_annotation():
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```